Parallel Bitonic Sort

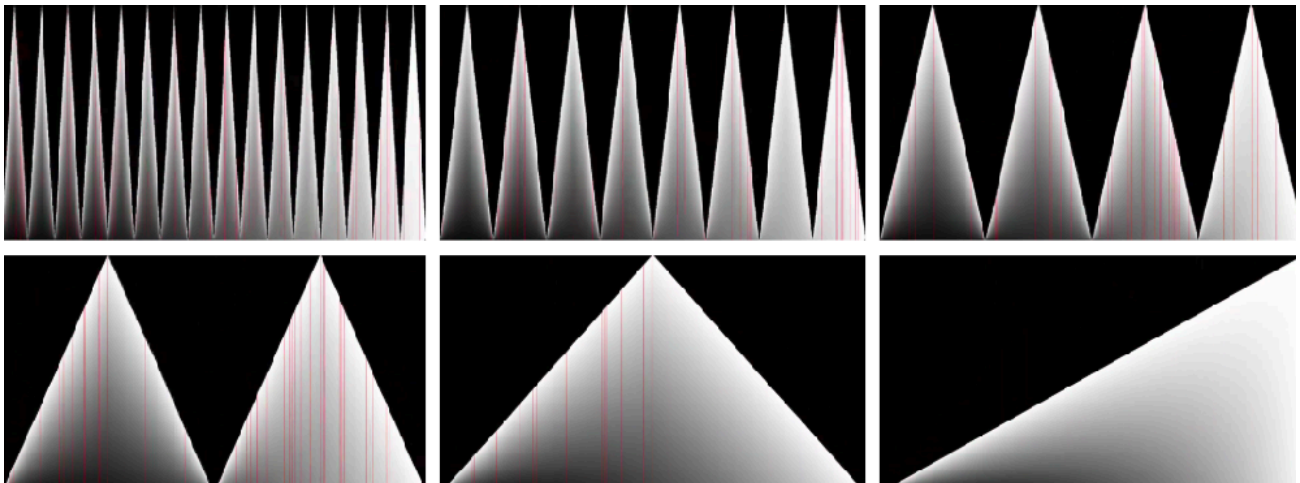
Sistemi di Elaborazione Accelerata M

A.A. 2025/2026

Gabriele Doti - 0001245897

Bitonic Sort

Algoritmo di ordinamento con complessità $n \log^2(n)$ che sfrutta le sequenze bitoniche



Analisi preliminare del problema

Input e Output:

Array di valori interi di lunghezza N in input, stesso array ordinato in ordine crescente in output

Vincoli:

La lunghezza dell'array è pari ad una potenza di 2, ma può anche essere maggiore $\Longrightarrow$ utilizzo di padding

Operazioni computazionali richieste:

1. Suddividere l'array in sequenze bitoniche (prima crescente, poi decrescente)
2. Comparare coppie di elementi secondo uno schema preciso
3. Scambiare gli elementi se non rispettano l'ordine desiderato

Potenziale di Parallelizzazione:

- **Vantaggi:**

1. Numerosi confronti indipendenti per livello $\Longrightarrow$ alto parallelismo
2. Operazioni regolari su array contigui $\Longrightarrow$ memoria coalescente

Analisi preliminare del problema (ii)

- **Limitazioni:**

1. Sincronizzazione tra i livelli necessaria
2. Gestione della memoria condivisa e globale per ridurre conflitti e latenza

Strategia di Decomposizione CUDA:

1. **Mappatura dei dati ai thread**:

- Un thread per ogni confronto oppure per ogni swap

2. **Uso della memoria:**

- Caricare sottosequenze in shared memory per ridurre accessi globali

3. **Gestione delle fasi:**

- Ogni fase della rete bitonica deve essere completata prima di passare alla successiva

Obiettivi:

Analisi preliminare del problema (iii)

1. Implementazione parallela: sviluppare versioni del Bitonic Sort ottimizzate sia su GPU (CUDA) sia su CPU utilizzando SIMD intrinsics, per sfruttare parallelismo massivo e vettorializzazione
2. Analisi delle prestazioni: confrontare le prestazioni delle versioni parallele rispetto a una implementazione seriale del Bitonic Sort, valutando velocità, scalabilità e utilizzo della memoria
3. Confronto con algoritmi commerciali/ottimizzati: valutare l'efficienza del Bitonic Sort parallelo rispetto ad algoritmi di ordinamento più efficienti come il Radix Sort (sia seriale sia SIMD), evidenziando punti di forza e limiti nell'uso di Bitonic Sort in contesti reali
4. Ottimizzazione e profiling: identificare colli di bottiglia, ottimizzare l'uso della memoria condivisa (per CUDA) e dei registri SIMD

L'algoritmo seriale

```
static void bitonic_merge(unsigned int* arr, unsigned int low, unsigned int cnt, bool
dir) {
    if(cnt>1){
        unsigned int k = cnt/2;
        for(unsigned int i=low;i<low+k;i++)
            if(dir==(arr[i]>arr[i+k])) std::swap(arr[i],arr[i+k]);
        bitonic_merge(arr, low, k, dir);
        bitonic_merge(arr, low+k, k, dir);
    }
}

static void bitonic_sort_rec(unsigned int* arr, unsigned int low, unsigned int cnt,
bool dir) {
    if(cnt>1){
        unsigned int k = cnt/2;
        bitonic_sort_rec(arr, low, k, true);
        bitonic_sort_rec(arr, low+k, k, false);
```

L'algoritmo seriale (ii)

```
        bitonic_merge(arr, low, cnt, dir);
    }
}

void bitonic_sort(unsigned int* arr, unsigned int n) {
    bitonic_sort_rec(arr, 0, n, true);
}
```

L'algoritmo si compone di due fasi principali:

1. Dividere l'array in due metà:

- Sinistra ⟶ valori più piccoli (o grandi)
- Destra ⟶ valori più grandi (o piccoli)

2. Ordinare separatamente ciascuna metà

Implementazione SIMD Intrinsics

I registri estesi possono essere sfruttati per eseguire i confronti a distanza j, dovendo posizionare i valori minori nella prima metà e quelli maggiore nella seconda metà

3 Implementazioni:

- Registri estesi a 128 bit
  https://raw.githubusercontent.com/gadoti03/BitonicSortAccelerator/main/simd/simd_sort_v1.cpp
- Registri estesi a 256 bit
  https://raw.githubusercontent.com/gadoti03/BitonicSortAccelerator/main/simd/simd_sort_v2.cpp

  Problema: se la dimensione della partizione k è minore della dimensione del registro a 256 bit, i registri estesi non vengono utilizzati e l'elaborazione avviene con registri normali in sequenziale
- Soluzione idbrida $\longrightarrow$ Soluzione migliore
  https://raw.githubusercontent.com/gadoti03/BitonicSortAccelerator/main/simd/simd_sort_v3.cpp

  Soluzione: utilizzare i registri a 256 bit se $k \geq 256$, quindi usare i registri a 128 bit se $k \geq 128$

Implementazione SIMD Intrinsics (ii)

Viene fornita anche una versione di Radix Sort ottimizzata con SIMD intrinsics, al fine di confrontare le prestazioni tra le soluzioni CUDA di Bitonic Sort e la migliore implementazione possibile su CPU dell'ordinamento di numeri interi

```
for (i = 0; i < n; i+=integers_per_reg * 4) {
    // Zero the digits accumulator
    digits_accumulator = _mm256_setzero_si256();
    for (j = 0; j < 4; j++) {
        // Read 8 integers
        vec = _mm256_load_si256((__m256i*)&arr[i+j*integers_per_reg]);
        // Extract the relevant 4 bits
        digits = _mm256_and_si256(_mm256_srli_epi32(vec, shift),
_mm256_set1_epi32(mask));
        // Shift digits to their position in the accumulator
        digits = _mm256_slli_epi32(digits, j * bits_per_pass * (8 / bits_per_pass));
        // Accumulate digits
        digits_accumulator = _mm256_or_si256(digits_accumulator, digits);
```

Implementazione SIMD Intrinsics (iii)

```
    }
    // Update count array
    for (index = 0; index < histogram_size; index++) {
        // Compare digits with index
        cmp = _mm256_cmpeq_epi8(digits_accumulator, _mm256_set1_epi8(index));
        // Take the most significant bit of each byte to form a mask
        mask_cmp = _mm256_movemask_epi8(cmp); // Compact comparison results into a 32-
bit integer
        // Count number of set bits in mask_cmp
        count_set_bits = __builtin_popcount(mask_cmp); // POPCNT

        count[index] += count_set_bits;
    }
}
```

Implementazione in CUDA

Setup applicato in tutte le versioni

NVIDIA GeForce RTX 2080 Ti

- Compute Capability: 7.5 | SMs: 68
- Warp size: 32 | Max threads/block: 1024 | Max threads/SM: 1024
- Shared memory/block: 48 KB | per SM: 64 KB | Banks: 32, 4B/bank
- Global memory: 10.8 GB | Constant memory: 64 KB | L2 cache: 5.5 MB
- Max blocks/SM: 16 | Max warps/SM: 32

N = 1 << 28
k = lughezza sequenza bitonica
j = distanza del confronto

Implementazione in CUDA: v1

Implemenazione naive, senza considerare specifiche HW e ottimizzazioni:

- Il kernel è chiaramente Memory-Bound:
  - Ogni thread del blocco confronta il proprio elemento con il corrispondente ed esegue lo swap in memoria centrale
  - Se lo swap è già stato eseguito, non si esegue nuovamente (esegue lo swap solamente il thread con indice globale minore)
- Problematiche:
  - Utilizzo della memoria globale per la totalità delle operazioni
  - Solo uno dei due thread esegue lo scambio, introducendo branching aggressivo

```
__global__ void bitonicCompareSwap(int *data, int k, int j, int N) {

    unsigned int gid = blockDim.x * blockIdx.x + threadIdx.x;
    unsigned int ixj = gid ^ j;
```

Implementazione in CUDA: v1 (ii)

```
    if (ixj > gid) { // prevent double swap
        if ((gid & k) == 0) { // ascending phase
            if (data[gid] > data[ixj])
                swap(data[gid], data[ixj]);
        } else { // descending phase
            if (data[gid] < data[ixj])
                swap(data[gid], data[ixj]);
        }
    }
}
```

Criticità

1. Se $j < 32$, metà dei thread del warp sono inattivi
2. Se $j \geq 32$, i thread del warp sono tutti attivi o inattivi $\Longrightarrow$ lancio il doppio dei blocchi necessari
3. Assenza di shared memory

Implementazione in CUDA: v1 (iii)

Compute Throughput [%]	Memory Throughput [%]
23,75	83,83
23,97	81,72
23,17	82,86
23,40	82,33
15,70	84,90
15,73	84,47
16,06	85,09
16,46	85,01
17,26	84,21
26,48	83,42
23,79	83,83
23,98	81,71
23,17	82,85
23,39	82,32
15,43	83,74
15,70	84,89
15,73	84,44

Interpretazione:

- Elevato memory throughput indica saturazione del sottosistema di memoria
- Basso compute throughput indica limitato sfruttamento delle ALU $\Longrightarrow$ Il problema è Memory Bound
- Nonostante l'alta occupancy, l'elevata percentuale di No Eligible Warps evidenzia stall dovuti a latenze di memoria

Implementazione in CUDA: v1 (iv)

Occupancy	
Theoretical Occupancy [%]	100
Theoretical Active Warps per SM [warp]	32
Achieved Occupancy [%]	77,18
Achieved Active Warps Per SM [warp]	24,70

Scheduler Statistics	
Active Warps Per Scheduler [warp]	6,06
Eligible Warps Per Scheduler [warp]	0,10
Issued Warp Per Scheduler	0,09
No Eligible [%]	90,58
One or More Eligible [%]	9,42

Implementazione CUDA: v2

Introduzione Shared Memory

- Dimensione massima: 1024 * 4 Byte = 4KB
- Uso: non solo swap, ma ordinamento di sottosequenze intere di lunghezza limitata a ThreadPerBlocco (direzione specifica)

Kernel

- Kernel 1 (nuovo): ordina sottosequenze di lunghezza ThreadPerBlocco
  - Esempio: primi 128 valori $\Longrightarrow$ crescente, secondi 128 $\Longrightarrow$ decrescente
- Kernel 2: kernel precedente, utilizzato successivamente
- Funzionamento:
  - Il Kernel 1 ordina i blocchi iniziali creando sequenze bitoniche di lunghezza ThreadPerBlock
  - Successivamente, il Kernel 2 esegue il merge di queste sequenze per produrre l'ordinamento crescente

Ottimizzazione

- Controllo incrociato su tempi per scegliere NumThread x Blocco ottimale per i 2 kernel
- Risultato: Per il primo kernel, NumThread x Blocco = 1024 $\Longrightarrow$ massimizza uso shared memory $\Longrightarrow$ migliori prestazioni

Implementazione CUDA: v2 (ii)

```
bitonicCompareLocal<<<gridDim_shared, blockDim_shared>>>(d_data);

for (int k = THREADS_PER_BLOCK_SHARED_MEMORY<<1; k <= N; k <<= 1) {
    for (int j = k >> 1; j > 0; j >>= 1) {
        bitonicCompareGlobal<<<gridDim, blockDim>>>(d_data, k, j);
    }
}
```

GPU Speed Of Light Throughput — All

Compute (SM) Throughput [%]	49,83	Duration [ms]	85,25
Memory Throughput [%]	49,83	Elapsed Cycles [cycle]	116.385.504
L1/TEX Cache Throughput [%]	53,14	SM Active Cycles [cycle]	115.260.178,68
L2 Cache Throughput [%]	1,39	SM Frequency [Ghz]	1,36
DRAM Throughput [%]	3,86	DRAM Frequency [Ghz]	6,79

Analisi del kernel bitonicCompareLocal:

- Il Compute Throughput cresce significativamente rispetto al kernel bitonicCompareGlobal

Implementazione CUDA: v2 (iii)

Memory Workload Analysis	
Memory Throughput [Gbyte/s]	25,16
L1/TEX Hit Rate [%]	50,00
L2 Hit Rate [%]	49,99

Interpretazione L1:

- Ogni thread legge una sola volta dalla memoria globale
- Nessun riuso del dato in L1
- L1 hit rate ≈ 50%, coerente con accessi allineati e coalescenti ma senza riutilizzo

Implementazione CUDA: v3

Miglioramenti introdotti:

- Introduzione della shared memory per l'ordinamento di tutte le sottosequenze di lunghezza pari a THREADS_PER_BLOCK_SHARED_MEMORY, non solo la sequenza bitonica iniziale
- Accesso alla memoria globale e memoria condivisa per confronto e swap ridotto a una singola lettura, con salvataggio in variabili private (registri)
- Dimezzamento blocchi lanciati per bitonicCompareGlobal

```
bitonicCompareLocal_init<<<gridDim_shared_init, blockDim_shared_init>>>(d_data, 0);
for (int k = THREADS_PER_BLOCK_SHARED_MEMORY_init<<1; k <= N; k <<= 1) {
    for (int j = k >> 1; j >= THREADS_PER_BLOCK_SHARED_MEMORY; j >>= 1)
        bitonicCompareGlobal<<<gridDim, blockDim>>>(d_data, k, j);
    int shift = 0;
    int tmp_k = k;
    while (tmp_k > THREADS_PER_BLOCK_SHARED_MEMORY) { tmp_k >>= 1; shift++; }
    bitonicCompareLocal<<<gridDim_shared, blockDim_shared>>>(d_data, shift);
}
```

Implementazione CUDA: v3 (ii)

```
__global__ void bitonicCompareGlobal(int *data, int k, int j) {
    int temp = j / THREADS_PER_BLOCK;
    int new_blockIdx = (blockIdx.x % temp) + (blockIdx.x / temp) * temp * 2;

    unsigned int gid = blockDim.x * new_blockIdx + threadIdx.x;
    unsigned int ixj = gid ^ j;

    int a = data[gid];   // register
    int b = data[ixj];   // register

    bool ascending = ((gid & k) == 0);

    if ((ascending && a > b) || (!ascending && a < b)) {
        data[gid] = b;
        data[ixj] = a;
    }
}
```

Implementazione CUDA: v3 (iii)

Compute (SM) Throughput [%]	81,87
Memory Throughput [%]	81,87
L1/TEX Cache Throughput [%]	97,94
L2 Cache Throughput [%]	4,82
DRAM Throughput [%]	13,40

L'impiego della shared memory aumenta l'efficienza complessiva del kernel, eliminando un collo di bottiglia predominante e consentendo una efficace mascheratura della latenza di memoria.

bitonicCompareLoc...	bitonicCompareLocal(i...	81,87	81,87	22
bitonicCompareGlo...	bitonicCompareGlobal...	15,63	85,00	16
bitonicCompareGlo...	bitonicCompareGlobal...	15,18	82,40	16
bitonicCompareGlo...	bitonicCompareGlobal...	13,18	71,27	16
bitonicCompareGlo...	bitonicCompareGlobal...	13,46	72,29	16
bitonicCompareGlo...	bitonicCompareGlobal...	13,72	72,64	16
bitonicCompareLoc...	bitonicCompareLocal(i...	81,88	81,88	22
bitonicCompareGlo...	bitonicCompareGlobal...	15,58	84,80	16
bitonicCompareGlo...	bitonicCompareGlobal...	15,64	85,01	16
bitonicCompareGlo...	bitonicCompareGlobal...	15,18	82,40	16
bitonicCompareGlo...	bitonicCompareGlobal...	13,19	71,31	16
bitonicCompareGlo...	bitonicCompareGlobal...	13,44	72,15	16
bitonicCompareGlo...	bitonicCompareGlobal...	13,72	72,64	16

Il beneficio ottenuto risulta significativo esclusivamente durante l'ordinamento delle sottosequenze di dimensione pari al massimo ThreadsPerBlock. Nelle fasi successive dell'algoritmo, l'accesso alla memoria globale torna a rappresentare il principale collo di bottiglia.

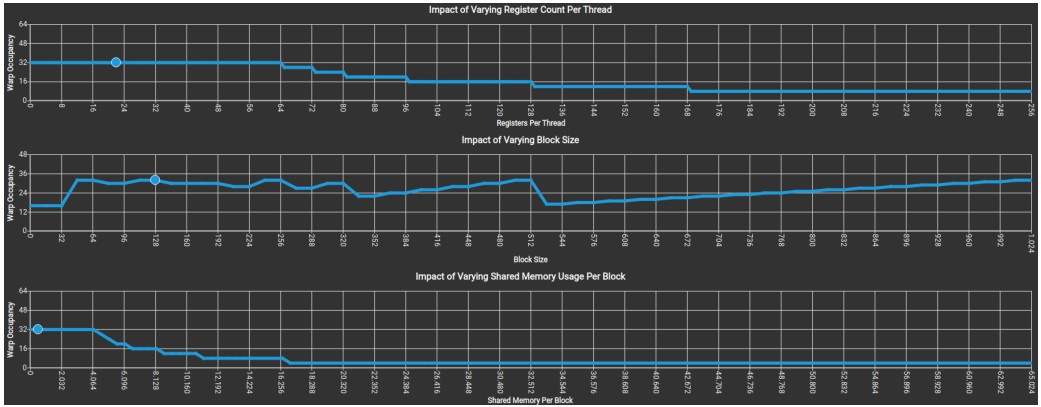
Implementazione CUDA: v3 (iv)

No Eligible [%]	57,42
One or More Eligible [%]	42,58

~40% di eligible warp è realistico perché:

- Stall sui caricamenti iniziali da global memory.
- Sincronizzazioni frequenti (syncthreads()).
- Divergenza nei branch dei warp.

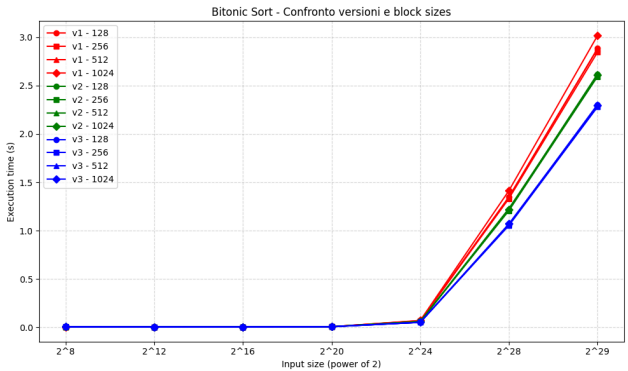
Implementazione CUDA: v3 (v)



Coerentemente all'analisi incrociata sui tempi, in cui è risultato che impostare a 128 il numero di ThreadPerBlock per il bitonicCompareLocal minimizzava il tempo di completamento dell'ordinamento, l'occupancy è massimizzata in corrispondenza di 128 ThreadPerBlock e 128 * 4B = 512B di shared memory per blocco

Implementazione CUDA: conclusioni

Confronto tra le tre versioni al variare della dimensione dell'input e della dimensione del blocco



- Beneficio della shared memory visibile per input > 2^{24} elementi
- Riduce gli accessi alla memoria globale, principale collo di bottiglia
- Ogni blocco, quando possibile, lavora su un chunk di dati in memoria condivisa, aumentando il throughput

Conclusioni finali

Confronto tra la versione CUDA e le versioni seriali e SIMD (intrinsics) di algoritmi di ordinamento commerciali, al variare della dimensione dell'input

Tempi di esecuzione algoritmi di sorting (ms) e rapporto con CUDA v3 Bitonic

	Serial QS	Serial MS	Serial RS	Serial BS	SIMD 128 bit BS	SIMD 256 bit BS	SIMD 256 bit RS	SIMD hybrid BS	CUDA v3 BS
2^8	0.051211	0.092568	0.025645	0.121797	0.043328	0.063947	0.006629	0.056784	5.54514
SpeedUp	0.01	0.02	0.00	0.02	0.01	0.01	0.00	0.01	1.00
2^12	1.15811	1.26758	0.114281	1.29675	1.05406	1.0288	0.110594	1.06803	4.39048
SpeedUp	0.26	0.29	0.03	0.30	0.24	0.23	0.03	0.24	1.00
2^16	11.5909	10.3915	1.63568	28.1073	10.8933	9.14542	0.693236	8.95321	3.94535
SpeedUp	2.94	2.63	0.41	7.12	2.76	2.32	0.18	2.27	1.00
2^20	152.601	168.147	101.097	604.826	132.046	151.285	12.5382	125.081	6.11782
SpeedUp	24.94	27.48	16.53	98.86	21.58	24.73	2.05	20.45	1.00
2^24	2930.49	3171.79	480.472	12153.5	3035.33	3277.23	203.49	2738.76	51.3959
SpeedUp	57.02	61.71	9.35	236.47	59.06	63.76	3.96	53.29	1.00
2^28	56767.2	66239.2	8526.63	276937	55251.9	64468.8	3207.78	52363.1	1059.15
SpeedUp	53.60	62.54	8.05	261.47	52.17	60.87	3.03	49.44	1.00
2^29	112804	135624	17285.9	576308	141436	145182	6962.28	124155	2281.49
SpeedUp	49.44	59.45	7.58	252.60	61.99	63.63	3.05	54.42	1.00

L'uso della GPU diventa conveniente per input superiori a 2^{16} elementi, ma lo speedup non aumenta proporzionalmente con la dimensione dell'input. Infatti, la shared memory viene utilizzata solo per

Conclusioni finali (ii)

ordinare sottosequenze di dimensione limitata, quindi all'aumentare dell'input totale ogni blocco sfrutta una porzione relativamente piccola della shared memory, riducendo lo speedup complessivo.